



Universidade de São Paulo

Instituto de Ciências Matemáticas e de Computação

Bacharelado em Ciências de Computação – 2022.1

SSC0240 – Bases de Dados

Projeto – Pokémons

Gerenciamento de experiência vivida por treinadores na realidade Pokemon

Docente: Prof^a Dr^a Elaine Parros Machado de Sousa

Estagiário PAE: André Moreira Souza

Giovanni Shibaki Camargo	11796444
João Pedro Gavassa Favoretti	11316055
Lucas Keiti Anbo Mihara	11796472
Lucas Pilla Pimentel	10633328
Rodrigo Lopes Assaf	11795530

Sumário

1 - Descrição do problema e dos requisitos de dados	4
1.1 - Restrições de Integridade	6
Possível inconsistência entre cpf (treinador) e nome (pet) na formação de times:	6
Possível inconsistência entre os ataques possíveis dos Pets e os ataques aprendidos por um Pet específico:	6
Ciclo: Treinador - Pokémon:	6
Ciclo: Treinador - Torneio:	7
Ciclo: Pokémon - Tipo - Ataque:	7
Ciclo: Pets - Ataque do Pet - Ataques específicos - Pokémon:	7
Ciclo: Pertences - Treinador - Batalha:	7
1.2 - Principais Operações	7
Treinador:	7
Mestre Pokemon:	8
2 - Modelo Entidade Relacionamento (MER)	9
3 - Alterações realizadas na segunda parte do projeto	10
4 - Modelo Relacional	11
5 - Justificativas para o Modelo Relacional	12
5.1. Relacionamento 1:N entre entidades Ataque e Tipo	12
5.2. Relacionamento 1:N entre entidades Competitiva e Torneio	12
5.3. Relacionamento entre entidade Pokémon e Tipo	13
5.4. Generalização/Especialização Mestre Pokémon	13
5.5. Agregação para criação da entidade Pet	14
5.6. Mapeamento de relacionamento entre Pet e Ataques Específicos	14
5.7. Mapeamento da agregação Pertences	15
5.8. Atributo multivalorado da entidade Mestre Pokémon	16
5.9. Atributo derivado da entidade Torneio	16
5.10. Mapeamento dos relacionamentos Time 1 e Time 2 entre Batalha e Pet	17
5.11. Auto-relacionamento da entidade Pokémon	17
6 - Alterações realizadas na terceira parte do projeto	18
7 - Consultas	18
7.1. Lista de pokémons, indicando os pokémons que um treinador possui.	18
7.2. Itens ordenados pela quantidade de vezes que foram utilizados.	18
7.3. Informações do treinador com mais vitórias em batalhas individuais.	19
7.4. Listar os treinadores que participaram de todos os torneios	20
7.5. Histórico de batalhas de um certo treinador	20
7.6. Informações do torneio com a maior quantidade de participantes.	21

7.7. Ranking dos treinadores baseado no saldo de vitórias em batalhas que ocorreram em um certo período.	21
7.8. Ataques mais utilizados nos pets	22
8 - Aplicação	22
8.1. Descrição Inicial	22
8.2. Inserção Realizada	23
8.3. Consulta Realizada	23
9 - Conclusão	24

1 - Descrição do problema e dos requisitos de dados

A aplicação idealizada pertencerá à realidade criada dentro do Mundo Pokémon, uma realidade fictícia. Em um escopo geral, ela visa ajudar a gerenciar algumas das experiências vividas por parte das pessoas dessa realidade, como: gerenciar suas batalhas, seus itens e seus pokémons. O usuário alvo da aplicação serão os Treinadores do Mundo Pokémon. O nome “Treinador” é a denominação para uma pessoa do Mundo Pokémon que possui o trabalho de interagir com os Pokémons (animais do mundo fictício) e treiná-los para torná-los mais fortes.

Dentre as funcionalidades que o usuário encontrará ao utilizar a aplicação estão: a possibilidade de criar novas batalhas ou entrar em batalhas existentes no sistema, consultar seu histórico de batalhas ou de outros treinadores, criar novos torneios, que consistem em uma série de batalhas chaveadas de forma que nas últimas batalhas só reste os melhores competidores, entrar em torneios já criados por outros treinadores, criar ou entrar em operações de trocas de itens ou pokémons com outro treinador, cadastrar os pokémons obtidos em sua mochila, que se refere à parte da interface que armazena os pokémons de cada treinador. Além disso, o sistema deve registrar as ações durante uma batalha, tais como o uso de itens, troca de pokémon, movimento de ataque ou saída da batalha.

De forma mais detalhada, abaixo serão listados todos os conceitos utilizados para descrever a aplicação: O **treinador** é uma pessoa comum no Mundo Pokémon, mas que coleciona **pokémons** e pode treiná-los para **batalhas** e/ou **torneios**. Cada treinador possui um conjunto de pokémons que pertencem a ele e estes são chamados de **pets**. Além disso, um treinador pode possuir um conjunto de **itens**, chamado de **pertences**. Ele também pode possuir um histórico de suas **batalhas** e **torneios** que participou. Para participar de uma batalha, o treinador pode utilizar somente um subconjunto dos pets. Cada treinador pode ser distinguido dos demais pelo seu CPF. Além disso, cada treinador possui nome, gênero, data de nascimento e sua cidade natal. O treinador pode se especializar e se tornar um **Mestre Pokémon**, para isso o treinador deve possuir um ginásio e várias especialidades tais como gelo, raio, água, fogo, etc. Além disso, quando o treinador se torna um Mestre Pokémon, ele pode organizar torneios. O treinador também pode cadastrar um pokémon como **descoberto** caso encontre ele na natureza ou em uma batalha.

Os **pokémons** são criaturas que os treinadores capturam, colecionam, nomeiam e usam em batalhas. Cada pokémon possui um número único para ser diferenciado dos demais. Quando um treinador captura um pokémon selvagem, o pokémon se torna seu pet e é armazenado na mochila do treinador. Cada criatura pode possuir 1 ou 2 **tipos** (fogo, água, elétrico, etc). Cada pokémon tem um conjunto de ataques possíveis que podem ser utilizados e podem ter no máximo 4 ataques por vez. Além do número, cada pokémon tem um nome e sua descrição.

Um **ataque** é um movimento que o **pokémon** pode realizar durante a batalha para ferir o pokémon oponente, sendo que cada ataque sempre tem exatamente 1 tipo. Este **ataque** pode ser diferenciado dos demais pelo seu nome. Cada ataque

contém uma descrição do **ataque**, além de sua potência, se ele é um ataque forte ou fraco, e a condição em que ele deixa ao adversário tal como paralizado, envenenado, queimado, e etc.

Os **ataques específicos** que cada pokémon pode possuir é um subconjunto de todos os ataques existentes no Mundo Pokémon. Isso significa que dentre todos os ataques existentes no Mundo Pokémon, cada espécie de pokémon pode ter somente alguns. Além disso, dentro dos quatro ataques que cada pokémon possui atualmente, todos devem estar dentro do conjunto de ataques específicos para aquela espécie.

Existem 18 **tipos** no Mundo Pokémon, dentre eles podemos citar *fire, water, grass, bug, fairy, dragon, etc.* Cada um desses tipos podem ser fortes, fracos ou imunes em relação a outros tipos. O **tipo** é conhecido univocamente por um nome, e tem uma cor associada a ele para a interface. Cada tipo possui certa vantagem sobre as demais, ou seja, um ataque do tipo A possui certo multiplicador de impacto sobre pokémons dos demais tipos. O valor desse multiplicador possui valores entre 0, 0.5, 1 e 2, e é utilizado como um fator na hora de calcular o poder de cada ataque. Tanto os pokémons quanto os ataques têm tipos, sendo que cada **ataque** tem apenas um **tipo** e o **pokémon** pode ter um ou dois **tipos**.

Um **pet** é um pokémon de certa espécie, mas que foi capturado por um treinador. Ao invés de possuir informações gerais da sua espécie, essa entidade possui informações específicas da criatura que foi capturada, como um nome, a natureza dele (se ele é calmo, agressivo, inquieto, etc.), o sexo dele, seu peso, sua altura, e sua habilidade especial. Um exemplo de habilidade especial é a característica de não receber dano de ataques de certo tipo. Além disso, o pokémon pode ter cores especiais, diferentemente de outros na sua espécie, e assim será chamado de *shiny*. A quantidade limite de pets que um treinador possui não é definida, podendo ter até mais de um pokémon da mesma espécie. Cada pet é identificado pelo seu treinador, pelo seu nome e pela sua espécie.

Os **Ataques do Pet** representam os ataques que certo pet possui. Esses ataques podem ser no mínimo 1 e no máximo 4 por pokémon. Esses ataques sempre devem pertencer ao conjunto de ataques que a espécie do pokémon pode possuir. Por fim, esses são os ataques, ou movimentos, que o pet poderá utilizar durante uma batalha.

As **batalhas** são interações entre dois times, cada time é formado por um treinador com no máximo seis de seus pets. Cada batalha possui data/hora, o resultado final, e uma descrição da batalha em si, assim como sua categoria. A batalha pode ser identificada dentre as demais pelos treinadores que participaram da batalha e a data/hora de acontecimento. Uma **batalha** também é associada aos **itens** que podem ter sido utilizados nela. O conceito de batalha pode ser de dois diferentes tipos: **Individual** e **Competitiva**. A batalha é identificada como Competitiva quando pertence à um torneio, caso contrário é identificada como Individual. De modo específico, a batalha individual possui o local de acontecimento. Do mesmo modo, a batalha competitiva possui a informação da etapa que

aconteceu durante o campeonato, como *eliminatória, oitavas, quartas, semi e final*, etc.

Os **torneios** envolvem diversas batalhas competitivas e são criadas por Mestres Pokémon. Dessa forma possuem múltiplos times. Com o chaveamento do torneio só restará um vencedor no final. Os torneios são identificados por seu local e data.

Os **itens** são todos os tipos de produtos que um treinador pode ter. Cada tipo de item fornece diferentes efeitos e podem ser utilizados para aumentar a vida dos pokémons, melhorar alguma habilidade do pokémon ou facilitar a captura do pokémon. Os itens são identificados unicamente pelo seu nome e possuem uma descrição do efeito que o item proporciona.

Os **pertences** são como bens de cada treinador. Cada pertence corresponde a um item em determinada quantidade. Cada pertence pode ser utilizado somente uma vez e possui registro de uso em batalha quando utilizado durante um combate.

1.1 - Restrições de Integridade

- **Possível inconsistência entre cpf (treinador) e nome (pet) na formação de times:**

A modelagem do banco de dados feito não garante que, na entidade Time, todos os pokémons de um certo time sejam do mesmo treinador. Tal restrição deve ser feita pela própria aplicação que deve deixar apenas os Pet cujo cpf do treinador coincida com a chave nome presente na entidade Time.

- **Possível inconsistência entre os ataques possíveis dos Pets e os ataques aprendidos por um Pet específico:**

A modelagem do banco de dados feito não garante que os nome de ataque presentes na entidade "Ataque dos Pets" sejam somente aqueles presentes na entidade "Ataques Específicos" de cada espécie de pokémon, pois o números (número do pokémon) vindos de 'Pet' e 'Ataques Específicos' podem não ser os mesmos. Dessa forma, a aplicação fica responsável de limitar, através da Entidade "Ataques Específicos" os ataques aprendidos por cada Pet.

- **Ciclo: Treinador - Pokémon:**

As entidades Treinador e Pokémon possuem dois relacionamentos que são dependentes entre si, pois, para que um treinador possua um pokémon ele deverá já ter descoberto o mesmo.

- **Ciclo: Treinador - Torneio:**

As entidades Treinador e Torneio possuem dois relacionamentos que são independentes entre si. Participar é uma ação totalmente diferente de organizar um torneio.

- **Ciclo: Pokémon - Tipo - Ataque:**

Este ciclo é essencial para o projeto, pois a entidade Pokémon precisa estar relacionada à entidade Ataque, pois todos os pokémons possuem certos ataques, independentemente do seu tipo, ou seja, pokémons de um certo tipo podem possuir ataques de outros tipos (por exemplo um pokémon do tipo fogo pode possuir um ataque do tipo terra). O relacionamento entre Pokémon e Tipo é necessário pois cada Pokémon possui de 1 a 2 tipos distintos. Por fim, o relacionamento entre Tipo e Ataque é necessário pois um ataque é de apenas um tipo específico.

- **Ciclo: Pets - Ataque do Pet - Ataques específicos - Pokémon:**

Neste ciclo, seria possível que um pet possuía, na entidade Ataque do Pet, ataques que não são de sua espécie, pois a modelagem não garante tal limitação. Dessa forma, a aplicação deverá, na hora de inclusão ou alteração dos Ataques do Pet de um pet específico, limitar que os ataques lá presentes sejam apenas os listados na entidade Ataques Específicos, que terá a listagem de todos os ataques possíveis que àquela espécie de pokémon.

- **Ciclo: Pertences - Treinador - Batalha:**

Neste ciclo, seria possível que um pertence de um treinador que não esteja participando da batalha seja utilizado. Assim sendo, a aplicação deve verificar se o treinador que é dono do pertence está participando daquela batalha.

1.2 - Principais Operações

Considerando os dois tipos diferentes de usuários, treinador e mestre pokémon, o sistema deve possuir as seguintes operações:

- **Treinador:**

- inserir, editar e deletar os pokémons que possui;
- editar os ataques de seus pokémons;
- inserir, editar e deletar os itens que possui em seus pertences;

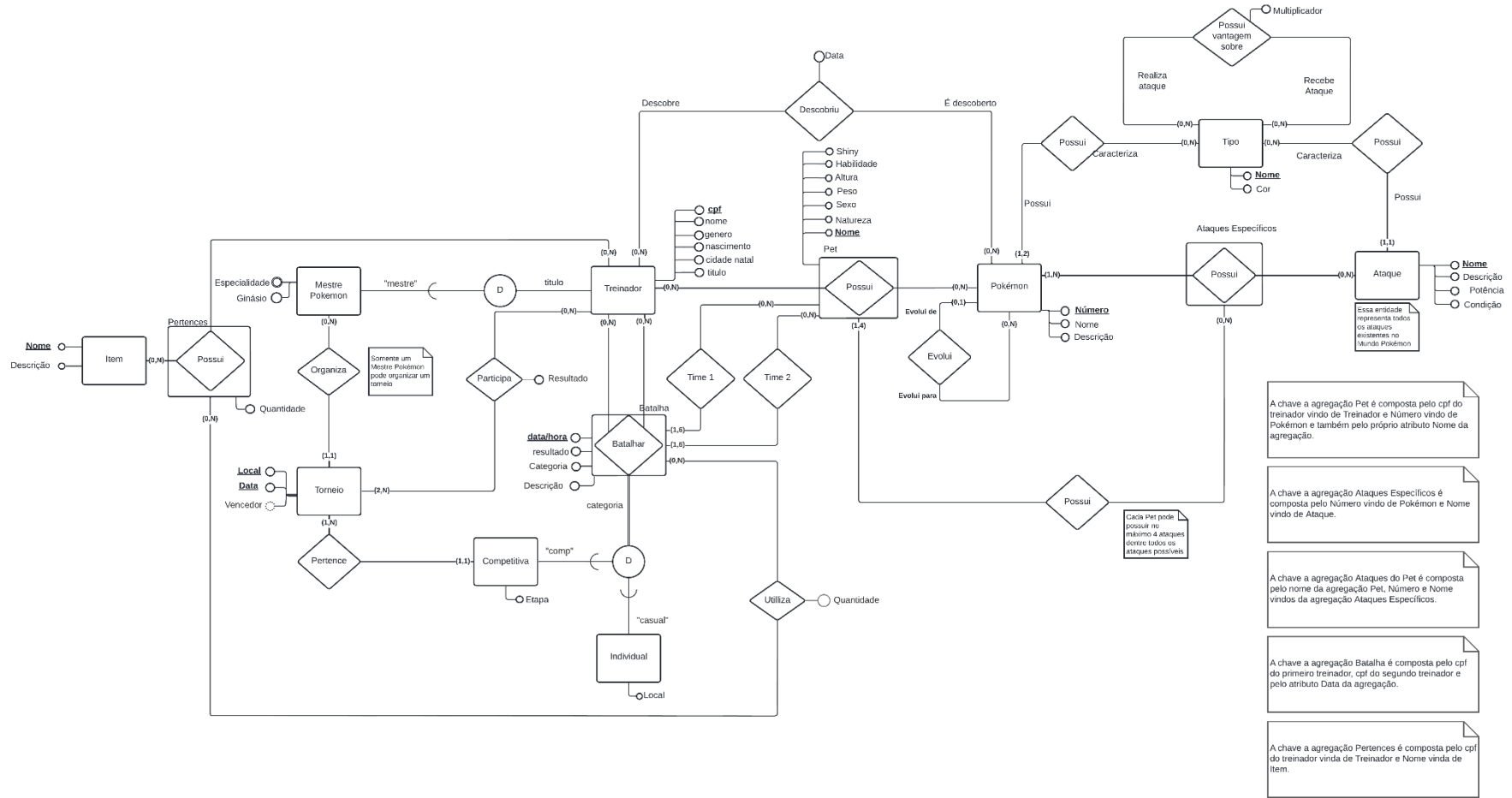
- busca por batalhas, podendo filtrar por àquelas em que participou, id de treinador e pokémons utilizados;
- busca por torneios já realizados, podendo filtrar por aqueles em que participou;
- busca por torneios futuros, podendo filtrar por data;
- inserir sua participação em um torneio;
- busca por espécies de pokémons descobertas, podendo filtrar por nome, tipo, ataque, número na pokédex (não é possível visualizar espécies de pokémons ainda não descobertas pelo treinador);
- busca por itens, podendo filtrar por nome e descrição;
- escolher o tipo de turno que irá realizar, se irá utilizar um ataque do pokémon atual, utilizar item que possui em seus pertences, ou trocar o pokémon atual por outro presente em seu time;
- escolher o ataque que será deferido pelo pokémon atual em batalha;
- escolher o item que será utilizado no pokémon atual em batalha;
- escolher o pokémon que irá trocar no lugar do pokémon atual em batalha;
- inserir, editar os pokémons presentes em seu time enquanto não há o início das batalhas.

- **Mestre Pokemon:**

Pode realizar todas as operações disponíveis para um treinador comum, porém possuem algumas operações adicionais voltadas especialmente para torneios. São elas:

- inserir, editar e deletar torneios pokémon;
- inserir, editar e deletar especializações que possui;
- inserir, editar e deletar o ginásio em que é mestre;

2 - Modelo Entidade Relacionamento (MER)



3 - Alterações realizadas na segunda parte do projeto

Para a segunda parte do projeto, por recomendação do monitor da disciplina e após discussão do grupo acerca do foco central e complexidade do projeto, retirou-se a parte de registro de Turnos em cada Batalha Competitiva ou Individual, o que fez com que alguns dos relacionamentos fossem alterados como por exemplo o relacionamento Possui entre as entidades “Ataques Específicos” e “Pet”. Além disso, arrumou-se algumas divergências de nomes de atributos entre a documentação do projeto e o Modelo Entidade Relacionamento desenvolvidos.

4 - Modelo Relacional

* NOT NULL



5 - Justificativas para o Modelo Relacional

A seguir estão listadas as justificativas tomadas para a criação do modelo relacional utilizando o modelo entidade relacionamento desenvolvido no tópico anterior.

5.1. Relacionamento 1:N entre entidades Ataque e Tipo

Solução adotada: Foi adicionado um parâmetro *tipo* dentro da relação Ataque com o sinal de NOT NULL. Nesse atributo foi utilizado uma chave estrangeira para o nome de um dos tipos cadastrados na tabela Tipo.

Vantagem: Como o relacionamento possui participação total, essa solução garante que esse campo nunca será nulo na hora de inserção em tabela.

Desvantagem: Caso o relacionamento não apresentasse participação total, essa alternativa não serviria e adicionaria o risco de manter em disco vários campos nulos. Porém, como esse cenário não ocorre, essa solução não possui nenhuma desvantagem evidente.

Solução alternativa: A principal solução alternativa para esse relacionamento é a criação de uma nova relação com dois atributos referenciando, por chave estrangeira, uma tupla na relação Ataque e outra na relação Tipo. Esse modo de solução não garante que toda entidade Ataque estará relacionada a um Tipo, por isso é menos apropriada do que a solução encontrada.

5.2. Relacionamento 1:N entre entidades Competitiva e Torneio

Solução adotada: Foi adicionado um parâmetro *torneio* dentro da relação Batalha_Competitiva com o sinal de NOT NULL. Nesse atributo foi utilizado uma chave estrangeira para o ID dos Torneios cadastrados na base.

Vantagem: A modelagem utilizada desse modo garante que toda entidade Competitiva estará relacionada com outra entidade do tipo Torneio.

Desvantagem: Utilizando a modelagem desse modo não é possível garantir a participação total de ambos os lados do relacionamento Pertence.

Solução alternativa: É possível realizar o mesmo mapeamento utilizando uma nova tabela para armazenar as ocorrências entre relacionamentos das duas

entidades. Entretanto, esse formato não permite garantir que toda Batalha Competitiva será associada a um torneio, o que é garantido utilizando a outra solução.

5.3. Relacionamento entre entidade Pokémon e Tipo

Solução adotada: Mesmo que o relacionamento Possui entre Pokémon e Tipo seja considerado N:N com participação total em um dos lados, semanticamente ele é considerado como dois relacionamentos 1:N sendo que somente um deles possui participação total. Foi adicionado dois atributos: *tipo_1* e *tipo_2*; dentro da relação Pokémon, com *tipo_1* possuindo a flag NOT NULL.

Vantagem: Essa solução garante que nenhum pokémon será inserido na tabela sem pelo menos 1 atributo *tipo* preenchido, o que seria um erro severo no contexto da aplicação. Assim, poucos pokémons não possuem dois tipos, dessa forma poucas tuplas irão armazenar o valor nulo. Além disso, utilizando atributos diretamente em relações para mapear um relacionamento N:N preveni o overhead criado por junções e indexação de uma nova tabela.

Desvantagem: Mesmo que poucos pokémons sejam cadastrados sem dois tipos, isso ainda representa uma desvantagem da solução encontrada.

Solução alternativa: A solução alternativa para essa modelagem se basearia na criação de uma nova tabela para mapear todos os relacionamentos entre entidades Pokémon e Tipo. Isso apresenta alguns problemas que inviabilizam a aplicação. Seria necessário realizar o algoritmo de junção muito frequentemente por se tratar de uma informação importante. Além disso, o custo em disco e performance para realizar a indexação baseado em duas chaves (string e inteiro) é maior do que na solução encontrada. Por fim, essa solução não garante que todo pokémon deve ter pelo menos 1 tipo associado e que ele deve ter no máximo 2 tipos.

5.4. Generalização/Especialização Mestre Pokémon

Solução adotada: A solução adotada para mapear a entidade generalizada Mestre Pokémon foi criar duas tabelas separadas para Treinador e Mestre Pokémon. Dentro da tabela Mestre Pokémon foi adicionado os atributos especializados da entidade e chaves estrangeiras para Mestre Pokémon foram direcionadas para sua chave primária.

Vantagem: Garante que os torneios sejam realizados apenas por treinadores classificados como Mestres Pokémon, fazendo também possível a busca por torneios realizados por um Mestre Pokémon específico.

Desvantagem: Adição de redundância em disco armazenado o CPF de um mestre pokémon duas vezes. Além disso, antes de realizar a inserção de um novo Mestre Pokémon, o treinador com as informações relacionadas deve ser inserido dentro da relação Treinador. Aumento da quantidade de processamento realizado ao inserir um novo Mestre Pokémon em tabela, porque mais chaves estrangeiras devem ser respeitadas para manter a integridade da base.

Solução alternativa: Seria possível armazenar os atributos do mestre pokémon diretamente na entidade Treinador, e depois realizar checagens em tempo de aplicação para a utilização desse Treinador como Mestre Pokémon ou somente como Treinador comum. Felizmente, todas essas checagens em aplicação podem ser solucionadas separando as entidades Mestre Pokémon e Treinador.

5.5. Agregação para criação da entidade Pet

Solução adotada: Foi realizado o mapeamento da agregação N:N diretamente com o mapeamento da agregação. As três chaves iniciais foram substituídas por um ID sintético.

Vantagem: Aumento da performance da base ao utilizar um ID sintético ao invés de dois atributos string e um inteiro. Diminuição no número de atributos trazidos ao utilizar uma chave estrangeira para uma tupla da relação Pet. Como o relacionamento não possui nenhum atributo, faz mais sentido mapear a agregação diretamente.

Desvantagem: Perda semântica ao utilizar um ID sintético.

Solução alternativa: Uma alternativa para esse mapeamento seria criar uma relação para efetivar o relacionamento entre Treinador e Pokémon, então criar uma relação com chave estrangeira para uma tupla dessa relação e depois mapear a agregação com os atributos da Entidade Pet.

5.6. Mapeamento de relacionamento entre Pet e Ataques Específicos

Solução adotada: Foi adicionado quatro atributos de ataque [1-4] dentro da relação Pet. Cada um desses atributos foram ligados à uma tupla de Ataques Específicos.

Vantagem: Como é ideal que um Pet tenha o máximo de ataques possíveis, poucas das tuplas terão valores nulos associados a esses atributos. Além disso, essa solução é efetiva na hora de garantir que todo Pet deve ter ao menos 1 ataque específico associado. Essa solução também limita a quantidade de ataques que um Pet pode ter, no máximo 4, o que é muito interessante no contexto do programa.

Desvantagem: Deve ser checado se existem ataques iguais, dentre os ataques possuídos por um Pet.

Solução alternativa: A solução alternativa para esse mapeamento seria criar uma nova tabela para mapear as incidências de ataques específicos para certo Pet. Essa solução não garante que um Pet terá no máximo 4 ataques, nem que um Pet deve ter ao menos um ataque. Isso também adiciona o overhead de indexação e junção entre tabelas.

5.7. Mapeamento da agregação Pertences

Solução adotada: A agregação do relacionamento entre as entidades Treinador e Item foi mapeada diretamente em uma nova tabela com o atributo quantidade associado. Além disso, foi utilizado uma chave sintética como chave primária da relação.

Vantagem: A utilização da chave sintética, um atributo do tipo inteiro, aumenta bastante a performance de busca e junção comparado à essa operação feita com dois atributos do tipo string. Além disso, como as tuplas formadas por essa agregação são utilizadas como chave estrangeira na tabela Pertence_Batalha, o id sintético possui relevância nesse caso. Permite que um treinador tenha inúmeros itens, o que é importante para a aplicação

Desvantagem: Como o relacionamento Possui não possui atributos próprios, não era tão ideal realizar o mapeamento do relacionamento separado ao mapeamento da agregação. Por isso esse método de mapeamento não possui tantas desvantagens sob esse cenário.

Solução alternativa: Uma solução alternativa seria impor uma restrição na quantidade de itens por treinador, dessa forma os itens que certo treinador possui seriam armazenados como atributos na tabela do Treinador. Como esse limite não existe e a quantidade de itens de diferentes jogadores não é constante, essa solução é inviável como modelagem.

5.8. Atributo multivalorado da entidade Mestre Pokémon

Solução adotada: Foi criada uma nova tabela para as especialidades de cada mestre pokémon cadastradas no sistema. Essa tabela possui uma chave estrangeira associando uma especialidade com um Mestre Pokémon cadastrado no sistema e um atributo especialidade do tipo string.

Vantagem: Essa solução permite que inúmeras especialidades sejam cadastradas, assim como permite que um Mestre Pokémon não tenha nenhuma especialidade.

Desvantagem: O principal problema dessa solução é o overhead criado ao criar uma nova tabela no SGBD. Uma estrutura de indexação deve ser armazenada em disco e o custo de junção e busca é adicionado ao procurar os dados de um Mestre Pokémon.

Solução alternativa: Fixar o número de especialidades “ativas” de um Mestre Pokémon e utilizar esse relacionamento como atributos dentro da tabela Mestre Pokémon

5.9. Atributo derivado da entidade Torneio

Solução adotada: Um atributo chamado de *vencedor* foi adicionado direto na relação Torneio.

Vantagem: A adição de um atributo vencedor direto na tabela Torneio facilita o acesso à essa informação caso seja consultado os dados de algum dos torneios realizados. Manter esse atributo atualizado é simples pois somente uma operação é realizada ao inserir o resultado do último combate do torneio. Além disso, poupa que todos os combates sejam checados até que o combate final seja encontrado e verificado o vencedor.

Desvantagem: Uma desvantagem dessa abordagem é que esse atributo permanece com valor nulo até que o torneio seja finalizado.

Solução alternativa: A alternativa para esse mapeamento seria não adicionar o atributo vencedor à tabela Torneio e realizar a operação de busca por todos os combates do torneio para verificar quem foi o vencedor.

5.10. Mapeamento dos relacionamentos Time 1 e Time 2 entre Batalha e Pet

Solução adotada: Para cada time, 6 atributos foram inseridos para indicar os Pets participantes, cada um com uma chave estrangeira para uma tupla da relação Pet. Totalizando a inserção de 12 atributos dentro da tabela Batalha.

Vantagem: Facilita a busca por Pets que participaram por uma Batalha em específico. Além disso, poucos valores são atribuídos com valor nulo porque em cada Batalha os treinadores levam consigo o máximo de pokémons que eles podem, que no caso são 6. Isso também define o número máximo de pokémons que um treinador pode levar em uma batalha.

Desvantagem: Necessidade de verificar se o Pet não é igual entre os Pets de um mesmo time e entre os Pets de ambos os times. Checagem se os Treinadores possuem tais Pets associados. Deve ser realizado várias operações de junção para obter toda a informação de uma Batalha.

Solução alternativa: A solução alternativa seria criar uma tabela separada com o relacionamento de cada Pet com o ID de uma Batalha. Isso iria garantir que um Pet não teria repetição mas ainda não iria garantir que um Pet é de determinado Treinador. Essa solução é inferior à adotada porque em termos de aplicação, a quantidade máxima de pokémons em uma batalha é mais crítico do que os demais problemas encontrados.

5.11. Auto-relacionamento da entidade Pokémon

Solução adotada: Foi criada uma tabela “Evolução” somente para armazenar as evoluções de pokémons. Essa tabela possui chave estrangeira para duas outras tuplas de dados pertencentes à tabela Pokémon.

Vantagem: Essa solução nos permite adicionar pokémons que não tenham evolução e também pokémons que tenham mais de uma evolução. Mas a principal vantagem dessa solução segue de que uma quantidade pequena de pokémons tem uma evolução. Dessa forma, caso tenha sido utilizado um parâmetro para indicar evolução diretamente na tabela Pokémon, então existiriam muitos campos nulos armazenados em disco.

Desvantagem: Deve ser garantido em aplicação que não pode haver nenhuma tupla que demonstre que um pokémon evolui para ele mesmo.

Solução alternativa: Como é usual, uma opção de mapeamento para um relacionamento 1:N é adicionar uma chave estrangeira diretamente na tabela

Pokémon com algum título relacionado à: “evolução_de”. E a partir desse atributo seria adicionada uma chave estrangeira diretamente para uma das tuplas existentes na tabela Pokémon.

6 - Alterações realizadas na terceira parte do projeto

Para a terceira parte do projeto, para os círculos pretos presentes no Modelo Relacional desenvolvido adicionou-se uma legenda explicando sua utilização e onde encontrar, na documentação do projeto, seu significado. Além disso, melhoramos a resolução da imagem do Modelo Relacional para que os sublinhados das chaves primárias e secundárias sejam exibidas de maneira correta conforme visualizado no LucidChart (plataforma em que desenvolvemos o MER e o Modelo Relacional).

7 - Consultas

7.1. Lista de pokémons, indicando os pokémons que um treinador possui.

```
SELECT PK.NUMERO,  
       PK.NOME,  
       CASE WHEN P.ID IS NULL THEN 'NÃO' ELSE 'SIM' END  
       AS POSSUI  
FROM POKEMON PK  
LEFT JOIN PET P ON PK.NUMERO = P.POKEMON  
                AND P.TREINADOR = '123-456-789-01'  
ORDER BY PK.NUMERO;
```

Essa funcionalidade é importante por permitir um treinador consultar os pokémons que já possui e aqueles que faltam. A consulta retorna o id e nome dos pokémons e indica se o treinador já possui, a lista é ordenada pelo número do pokémon.

7.2. Itens ordenados pela quantidade de vezes que foram utilizados.

```
SELECT I.NOME, NVL(SUM(PB.QUANTIDADE),0) AS QUANTIDADE  
FROM ITEM I  
LEFT JOIN PERTENCE P ON P.ITEM = I.NOME  
LEFT JOIN PERTENCE_BATALHA PB ON P.ID = PB.PERTENCE  
GROUP BY I.NOME  
ORDER BY QUANTIDADE DESC;
```

A consulta implementada permite listar todos os itens cadastrados na base de dados ordenados pela quantidade de vezes que foram utilizados em batalhas. Tal consulta retorna o nome e quantas vezes o item foi utilizado, indicando também aqueles que nunca foram usados. Para os treinadores, usuários da aplicação, é interessante saber quais itens são importantes.

Obs: a função NVL retorna zero para valores nulos.

7.3. Informações do treinador com mais vitórias em batalhas individuais.

```
SELECT
    T.CPF,
    T.NOME,
    T.GENERO,
    TO_CHAR(T.DATA_NASC, 'DD/MM/YYYY'),
    T.CIDADE_NATAL,
    T.TITULO
FROM BATALHA_INDIVIDUAL BI
JOIN BATALHA B
    ON B.ID = BI.BATALHA
JOIN TREINADOR T
    ON B.TREINADOR_1 = T.CPF OR B.TREINADOR_2 = T.CPF
GROUP BY T.CPF, T.NOME, T.GENERO, T.DATA_NASC,
    T.CIDADE_NATAL, T.TITULO
HAVING COUNT(B.ID) = (
    SELECT MAX(COUNT(B.ID))
    FROM BATALHA_INDIVIDUAL BI
    JOIN BATALHA B
        ON B.ID = BI.BATALHA
    JOIN TREINADOR T
        ON B.TREINADOR_1 = T.CPF OR B.TREINADOR_2 = T.CPF
    GROUP BY T.CPF
);
```

Esta consulta indica as informações do treinador que possui mais vitórias em batalhas individuais, retornando o CPF, nome, gênero, data de nascimento, cidade natal e título do treinador.

Obs: O resultado pode retornar mais de uma tupla caso haja mais do que um treinador com a maior quantidade de vitórias.

7.4. Listar os treinadores que participaram de todos os torneios

```
SELECT T.CPF, T.NOME FROM TREINADOR T
WHERE NOT EXISTS (
    (SELECT ID FROM TORNEIO)
    MINUS
    (SELECT TORNEIO
     FROM TREINADOR TT
     JOIN BATALHA B
     ON TT.CPF = B.TREINADOR_1 OR TT.CPF = B.TREINADOR_2
     JOIN BATALHA_COMPETITIVA BC
     ON B.ID = BC.BATALHA
     WHERE TT.CPF = T.CPF)
);
```

Essa consulta retorna uma lista dos treinadores que participaram de todos os torneios já criados, o resultado é uma lista com CPF e nome dos treinadores. Permite verificar os treinadores mais engajados.

7.5. Histórico de batalhas de um certo treinador

```
SELECT R.OPONENTE,
       TO_CHAR(R.DATA, 'DD/MM/YYYY') AS DATA,
       R.DESCRICAO,
       R.RESULTADO
FROM ((SELECT B.TREINADOR_2 AS Oponente,
             B.DATA, B.DESCRICAO,
             CASE B.RESULTADO WHEN 'TREINADOR 1'
              THEN 'VITORIA' ELSE 'DERROTA' END AS RESULTADO
      FROM BATALHA B
      WHERE B.TREINADOR_1 = '123-456-789-01')
 UNION
 (SELECT B.TREINADOR_1,
       B.DATA, B.DESCRICAO,
       CASE B.RESULTADO WHEN 'TREINADOR 2'
        THEN 'VITORIA' ELSE 'DERROTA' END
      FROM BATALHA B
      WHERE B.TREINADOR_2 = '123-456-789-01')) R
ORDER BY DATA DESC;
```

Esta consulta é importante por permitir que o usuário acesse o histórico de batalhas de um certo treinador. A consulta retorna informações das batalhas, indicando o oponente, data, descrição da batalha e se obteve 'VITÓRIA' ou 'DERROTA', com os dados ordenados de maneira decrescente

pela data da batalha. Nesse caso é preciso considerar que o treinador pode ser cadastrado como treinador 1 ou 2 em cada Batalha.

7.6. Informações do torneio com a maior quantidade de participantes.

```
SELECT T.ID, T.LOCAL, T.DATA, T.VENCEDOR, T.ORGANIZADOR,  
COUNT(TT.TREINADOR) AS QT_PARTICIPANTES  
FROM TREINADOR_TORNEIO TT  
JOIN TORNEIO T ON TT.TORNEIO = T.ID  
GROUP BY T.ID, T.LOCAL, T.DATA, T.VENCEDOR, T.ORGANIZADOR  
HAVING COUNT(TT.TREINADOR) = (  
    SELECT MAX(COUNT(TT.TREINADOR))  
    FROM TREINADOR_TORNEIO TT  
    GROUP BY TT.TORNEIO  
);
```

Com esta consulta é possível obter informações do torneio mais popular, ou seja, que obteve mais participantes. O resultado da consulta inclui: ID do torneio, local, data, vencedor, organizador e quantidade de participantes.

7.7. Ranking dos treinadores baseado no saldo de vitórias em batalhas que ocorreram em um certo período.

```
SELECT R.TREINADOR, SUM(R.RESULTADO) AS SALDO  
FROM ((SELECT B.TREINADOR_1 AS TREINADOR,  
    CASE B.RESULTADO WHEN 'TREINADOR 1' THEN 1 ELSE -1 END  
    AS RESULTADO,  
    B.DATA  
    FROM BATALHA B)  
    UNION ALL  
    (SELECT B.TREINADOR_2,  
    CASE B.RESULTADO WHEN 'TREINADOR 2' THEN 1 ELSE -1 END,  
    B.DATA  
    FROM BATALHA B)) R  
WHERE R.DATA BETWEEN '01/01/2020' AND '01/06/2020'  
GROUP BY R.TREINADOR  
ORDER BY SALDO DESC;
```

Na aplicação, é interessante que tenha um ranking dos treinadores para que os usuários possam competir e disputar pelas melhores posições, assim como identificar os melhores em certo período. Assim, esta consulta indica quais foram os melhores treinadores para o período desejado. A

consulta retorna uma lista com o CPF dos treinadores e o saldo de vitórias para o período especificado. Para essa query é considerado que um treinador pode ser cadastrado como treinador 1 ou 2.

7.8. Ataques mais utilizados nos pets

```
SELECT A.NOME, COUNT(R.POKEMON) AS QT_UTILIZACOES
FROM ((SELECT P.POKEMON, P.ATAQUE_1 AS ATAQUE FROM PET P)
      UNION ALL
      (SELECT P.POKEMON, P.ATAQUE_2 FROM PET P)
      UNION ALL
      (SELECT P.POKEMON, P.ATAQUE_3 FROM PET P)
      UNION ALL
      (SELECT P.POKEMON, P.ATAQUE_4 FROM PET P)) R
JOIN ATAQUE_ESPECIFICO AE ON R.ATAQUE = AE.ID
RIGHT JOIN ATAQUE A ON AE.ATAQUE = A.NOME
GROUP BY A.NOME
ORDER BY COUNT(*) DESC;
```

Esta consulta retorna uma lista de todos os ataques, e para cada ataque a quantidade de vezes que aparece nos pets. A funcionalidade implementada nessa consulta permite com que os treinadores se preparem melhor para as batalhas e conheçam os ataques importantes.

8 - Aplicação

8.1. Descrição Inicial

Na aplicação desenvolvida utilizou-se o SGBD relacional Oracle disponibilizado pela professora da disciplina, além da linguagem Java na versão Oracle OpenJDK 17.0.2. De forma a possibilitar a conexão entre a aplicação e a base de dados, fez-se uso do Driver Oracle JDBC. O código executável se encontra no diretório SCC0240_Projeto_BD\out\artifacts\PokemonDB.jar. Para executar, basta realizar o seguinte comando: <java -jar PokemonDB.jar>.

Para as duas ações temos uma classe Querries com as duas possíveis Querries preparadas (src/database/Querries.java):

```
package database;

public final class Querries {
    private Querries() { }
```

```

    public static String GetPokemon = "SELECT * FROM POKEMON WHERE
    NUMERO = NVL(?, NUMERO) AND UPPER(NOME) LIKE UPPER(NVL(?, NOME)) AND
    ((UPPER(TIPO_1) = UPPER(NVL(?, TIPO_1)) AND UPPER(TIPO_2) = UPPER(NVL(?,
    TIPO_2))) OR (UPPER(TIPO_1) = UPPER(NVL(?, TIPO_1)) AND UPPER(TIPO_2) =
    UPPER(NVL(?, TIPO_2)))) ORDER BY NUMERO";

    public static String InsertTrainer = "INSERT INTO TREINADOR (CPF,
    NOME, GENERO, DATA_NASC, CIDADE_NATAL, TITULO) VALUES (?, ?, ?, ?, ?,
    ?)";
}

```

Depois usamos esta classe para realizar as queries se preciso com o auxílio da biblioteca sql do java para preparar os parâmetros/atributos das queries, como no exemplo abaixo (src/action/insertTrainer.java):

```

public void insert() throws SQLException
{
    String sql = Queries.InsertTrainer;
    PreparedStatement stmt;

    stmt = db.prepareStatement(sql);
    stmt.setString(1, trainer.cpf);
    stmt.setString(2, trainer.name);
    stmt.setString(3, Character.toString(trainer.gender));
    stmt.setString(4, trainer.birthDate);
    stmt.setString(5, trainer.homeTown);
    stmt.setString(6, trainer.title);

    stmt.execute();
}

```

8.2. Inserção Realizada

Para a funcionalidade de cadastro, a query selecionada foi a de adição de um novo treinador no sistema, em que a interface verifica se os dados estão no formato certo, e há a verificação para não inserir dois treinadores com mesma chave (CPF), aceitando valores nulos para atributos não chave.

8.3. Consulta Realizada

Para a funcionalidade de consulta, a query selecionada foi a de seleção de pokémons no sistema, em que a interface aceita buscas por número, nome, e tipos do pokémon, verificando substrings no caso do nome. Pode-se aplicar mais de um filtro por vez.

9 - Conclusão

O desenvolvimento de um projeto envolvendo os principais tópicos da disciplina proporcionou vários desafios para a equipe. Desde o desenvolvimento da ideia de um projeto multimídia até a criação da aplicação final, vários ajustes na ideia inicial tiveram que ser realizados para garantir que o sistema todo estaria robusto.

O tema que começou baseado no interesse de alguns alunos do grupo terminou em um projeto que ajudou todos da equipe a fixar os conhecimentos vistos em aula e exemplificar a importância deles.

A tradução da ideia inicial do projeto para o Modelo de Entidades e Relacionamentos (MER) foi o primeiro grande desafio que tivemos como equipe. Diferentes soluções foram pensadas para o mesmo problema e tivemos que chegar na solução final a partir do problema como um todo. Esse foi um dos maiores desafios que tivemos durante todo o trabalho porque foi o primeiro instante que tivemos que pensar na aplicação fora da teoria e mais na prática, a partir da divisão de cada elemento em suas entidades e relacionamentos.

A partir disso criar o Modelo Relacional se mostrou inicialmente fácil porque já tínhamos discutido grande parte dos problemas ao construir o MER. No entanto, houveram algumas discussões sobre como poderíamos organizar o diagrama de forma que fosse facilmente interpretado por alguém que não tivesse sido apresentado à ideia antes.

Durante a construção do banco de dados, com a criação de esquema e inserção de dados, não houve tanto trabalho. Nesse momento o principal problema foi a repetição de trabalho para conseguir inserir dados o suficiente para gerar boas consultas posteriormente. O processo de criação do script de consultas foi muito mais interessante do que havíamos imaginado antes. Foi necessário pensar em consultas que fariam sentido no contexto da aplicação mas que também fosse algo desafiador de otimizar.

Para nós a maior dificuldade na aplicação foi a utilização de uma interface gráfica ao invés de usar o próprio terminal, e também a sanitização dos dados no formato certo, como no caso de CPF e data.

Por fim, projetar uma base a partir da criação de todos os modelos foi uma experiência fantástica e que será necessária para qualquer aplicação que formos construir no futuro. Além de saber utilizar os componentes da forma correta, é necessário pensar de modo que a modelagem faça sentido, e isso se mostrou muito interessante para nosso grupo.